



S.G. Ganesh

Understanding the *sizeof* Operator

In C/C++, the *sizeof* operator is used extensively. In this column, we'll look at some simple and interesting aspects of the *sizeof* operator in a Q&A format.

Assume that the *sizeof* pointer size and *int* is 4.

1) Which is the only 'compile-time' operator in C?

2) Can *sizeof* return zero?

3) What does the following statement print?

```
printf("%d", sizeof(printf("hello")));
```

4) What will be the output of the statement below?

```
printf("%d %d", sizeof("1"), sizeof("1"));
```

5) What does the following program print?

```
#define SIZE(arr) (sizeof(arr)/sizeof(arr[0]))
```

```
int size(int arr[]){
    return (sizeof(arr)/sizeof(arr[0]));
}
```

```
int main(){
    int arr[10];
    printf("%d %d", SIZE(arr), size(arr));
}
```

6) What is the output of the following statement?

```
printf("%d %d %d", sizeof(void *),
       sizeof(int *), sizeof(int **));
```

The answers

1) A compile time operator means that the operator is evaluated entirely at compile-time. For the *sizeof* operator, its use results in a constant integer value (technically *size_t* type) at compile-time. So, the only 'compile-time' operator in C is *sizeof* (except for variable length arrays where *sizeof* evaluates the size at runtime).

2) No. *sizeof* always returns the size of a variable or type, and so it cannot be zero. If you give an incomplete type (say a struct declaration or void type) as an argument to *sizeof*, it will result in a compiler error.

3) It prints 4. How about printing 'hello'? No, it does not print that! If *sizeof* is given a function call as argument, it returns the size of the return type. The return type of *printf* is *int* (*printf* returns the number of characters it successfully printed). Since we assumed *sizeof* *int* to be 4 here, the outer *printf* prints 4!

4) The output will be 1 2. For *sizeof* "1", the argument is a character '1', and size of a character is 1 in C. For *sizeof* "1 ", the argument "1 " is a string literal, which has a terminating character '\0' in it; so the size of the string literal is 2.

5) It prints 10 1. C/C++ does not provide a direct way to find the size of an array. One widely-used 'trick' is to divide the size of the array by the size of the element of the array. The macro *SIZE* does precisely that.

Given the declaration "int arr[10];", *sizeof*(arr) is $10 * 4 = 40$ bytes, since the array of size 10 is capable of holding 10 integers. The expression *sizeof*(arr[0]) is nothing but *sizeof*(int), which is 4. So the expression *sizeof*(arr)/*sizeof*(arr[0]) gives the value 10, which is the size of the array.

Coming to the *size* function, in the signature *int size(int arr[])*, the argument is "int arr[]". The argument is equivalent to the signature *int size(int *arr)*! In C, we cannot pass whole arrays—we can only pass the base address of an array. So, in the *size* function, the expression "sizeof(arr)/sizeof(arr[0])" results in 1 assuming the size of the pointer and *int* is the same (4 bytes)!

6) The output is 4 4 4. The sizes of pointer types are the same, irrespective of the type of the pointer and the level of indirection! 

About the author:

S G Ganesh is a research engineer in Siemens (Corporate Technology). His latest book is "Cracking the C, C++ and Java Interview", published by Tata McGraw-Hill. You can reach him at sgganesh@gmail.com.